MVC Forms, Validation & Data Access with EF Core 9

Day 2

Agenda for Day 2

- HTML Forms in MVC (GET vs. POST, Tag Helpers)
- Model Binding
- Data Annotations Validation
- Displaying Validation Errors
- Entity Framework Core 9 Integration with MVC
- Performing CRUD Operations in MVC
- Lab Assignment

HTML Forms

- The **`<form>`** Element: used to create a container for different types of input controls. Its primary
- **Purpose** :collect information from a user and send it to a server for processing.

HTML Forms

- **Key Attribute**:
  - `action` : specifies the *URL* of the server-side script or *endpoint* that will process the form data when it's submitted.
  - `method` : defines the *HTTP method* to be used when submitting the form data.
    - `GET` : Appends the form data to the URL as a query string
    - `POST`: Sends the form data within the body of the HTTP request
  - `target` :
    - `_self`
    - `_blank`

HTML Forms in MVC

```
<!-- Example of a GET form -->
<form action="/Search" method="GET">
    <input type="text" name="query" placeholder="Search...">
    <button type="submit">Search</button>
</form>

<!-- Example of a POST form -->
<form action="/Product/Create" method="POST">
    <input type="text" name="ProductName" placeholder="Product Name">
    <input type="number" name="Price" placeholder="Price">
    <button type="submit">Add Product</button>
</form>
```

HTML Forms in MVC

- **Purpose**: Collect user input.
- `method="GET"`:
  - Appends form data to the URL as query string.
  - Suitable for search forms, non-sensitive data.
  - Idempotent (can be bookmarked, refreshed).
- `method="POST"`:
  - Sends form data in the request body.
  - Suitable for sensitive data, creating/updating resources.
  - Not idempotent (refreshing might resubmit).

Form Tag Helpers

- **`<form asp-action="ActionName" asp-controller="ControllerName">`** : Generates the `action` and `method` attributes.
- **`<input asp-for="PropertyName" class="form-control" />`**:
  - Generates `id`, `name`, `value` attributes.
  - Sets `type` based on `DataType` attribute (e.g., `[DataType(DataType.EmailAddress)]`).
  - Connects to validation.

Form Tag Helpers

- **`<select asp-for="PropertyName" asp-items="SelectList">`**: For dropdowns.
- **`<textarea asp-for="PropertyName">:`** For multi-line text input.

Example

- .cshtml

```
<input asp-for="Name" class="form-control" />
```

- .html

```
<input class="form-control" type="text" ="trudata-vale" data-val-
required="The Name field is required." id="Name" name="Name" value="" />
```

Model Binding

- **Concept**: The process by which ASP.NET Core maps incoming HTTP request data (from forms, route data, query strings) to C# action method parameters or complex models.
- **How it works**: Looks for matching names between request data and model properties.
- **Example**: Form field `name="Product.Name"` maps to `Product.Name` property.
- `ModelState.IsValid` :checks if model binding was successful AND all validation rules passed

```
if (ModelState.IsValid)
    {
    }
```

Model Binding

- Binding to primitive types

```
HTTP GET
http://localhost:52201/Products/Edit/1
http://localhost:52201/Products/Edit?id=1
```

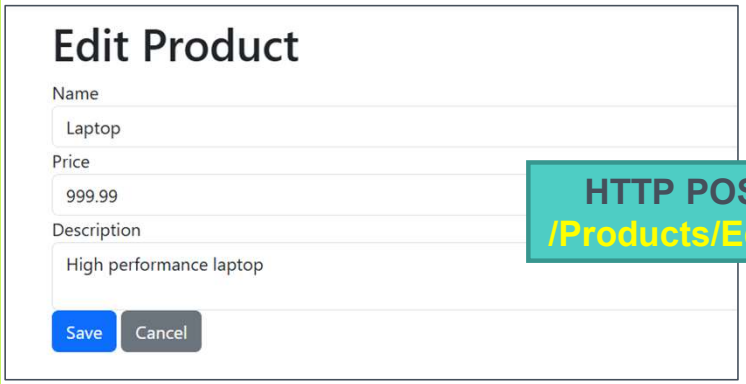
```
HTTP GET
http://localhost/Student/Edit?id=1&name=John
```

```
// GET: Products/Edit/5
public ActionResult Edit(int id)
{
```

```
// GET: Student/Edit/5
public ActionResult Edit(int id, string name)
{
```

Model Binding

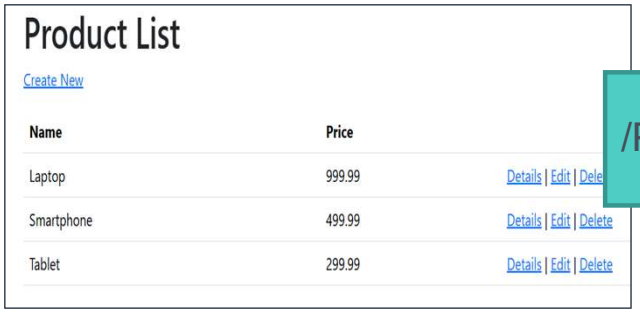
- Binding to Complex type



HTTP POST
/Products/Edit/1

```
[HttpPost]
public IActionResult edit(Product product)
{
    if (ModelState.IsValid)
    {
        var oldProduct =
_productRepository.GetProductById(product.Id);
        oldProduct.Name = product.Name;
        oldProduct.Description =
                    product.Description;
        oldProduct.Price = product.Price;
        return RedirectToAction("Index");
    }
    return View(product);
}
```

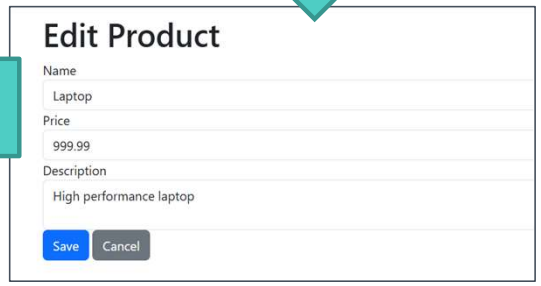
Edit View



HTTP GET
/Products/Edit?id=5
/Products/Edit/5

```
[HttpGet]
public IActionResult Edit(int id)
{
    var product=
_productRepository.GetProductById(id);
  return View(product);
}
```

Render



HTTP POST
/Products/Edit

```
[HttpPost]
public IActionResult Edit(Product product)
{
var oldProduct =
_productRepository.GetProductById(product.Id);
        oldProduct.Name = product.Name;
        oldProduct.Description =
                    product.Description;
        oldProduct.Price = product.Price;
        return RedirectToAction("Index");
```

Data Annotations Validation

- **Purpose**: Define validation rules directly on your model properties.
- **Attributes**:
  - `[Required]`: Field cannot be empty.
  - `[StringLength(max, min)]`: String length constraints.
  - `[Range(min, max)]`: Numeric range constraints.
  - `[EmailAddress]`, `[Url]`, `[Phone]`: Format validation.
  - `[Compare("OtherProperty")]`: Compares two properties (e.g., password confirmation).
  - `[RegularExpression("pattern")]`: Custom regex validation.
  - `[Display(Name = "Friendly Name")]`: For display labels.

Displaying Validation Errors

- `asp-validation-for="PropertyName"`:
  - Generates a `<span>` element.
  - Displays the validation error message for the specified property.
- `asp-validation-summary="ModelOnly"`:
  - Displays all model-level errors (errors not tied to a specific property).
    - (e.g., "Passwords do not match")
  - `All`: Displays all errors (model and property).
- `ModelState.IsValid`:
  - A Boolean property in the controller.
  - Returns `true` if all validation rules pass, `false` otherwise.
  - Always check `ModelState.IsValid` in POST actions.

`[ValidateAntiForgeryToken]`

- **Purpose** : Prevents Cross-Site Request Forgery (CSRF) attacks.
- **How it works**:
  - Generates a hidden form field with a unique, cryptographically secure token.
  - On form submission, the server validates this token.
  - If the token is missing or invalid, the request is rejected.
- **Usage**:
  - Add `[ValidateAntiForgeryToken]` attribute on POST actions.
  - `<input type="hidden" name="__RequestVerificationToken" />` (generated by `asp-action` on `<form>`)

Entity Framework Core 9 Integration

- **Review**: EF Core is an ORM that simplifies database interactions.
- **Configuration**:
  - Add `Microsoft.EntityFrameworkCore.SqlServer` and `Microsoft.EntityFrameworkCore.Tools` NuGet packages.
  - Configure `DbContext` in `Program.cs` using `builder.Services.AddDbContext<YourDbContext>( ... )`.
  - Set up connection string in `` `appsettings.json` ``.
- **Migrations**: Use `Add-Migration` and `Update-Database` to manage schema changes.

Entity Framework Core 9 Integration

- Apply migrations on startup

```
using (var scope = app.Services.CreateScope())
{
  var dbContext =scope.ServiceProvider.GetRequiredService<CompanyDbContext>();
  dbContext.Database.Migrate();
}
    app.Run();
```

Performing CRUD Operations in MVC (with EF Core)

- **Inject** `DbContext` :

```
private readonly ApplicationDbContext _context;
public MovieController(ApplicationDbContext context) { _context = context; }
```

- **Create (POST)**:

```
_context.Movies.Add(movie);
await _context.SaveChangesAsync();
```

Performing CRUD Operations in MVC (with EF Core)

- **Read (GET)**:

```
await _context.Movies.ToListAsync();
await _context.Movies.FirstOrDefaultAsync(m => m.Id == id);
```

- **Update (POST):**

```
_context.Movies.Update(movie); // Or modify tracked entity
await _context.SaveChangesAsync();
```

Performing CRUD Operations in MVC (with EF Core)

- **Delete (POST):**

```
_context.Movies.Remove(movie);
await _context.SaveChangesAsync();
```

- **Always use `async` / `await` for database operations.**

Repository Pattern (Adaptation for MVC)

- **Review**: Abstraction layer for data access.
- In MVC: You can still use the generic `IRepository<T>` and `Repository<T>` from the Web API course.
- **Pros**: Consistent data access, testability.
- **Cons**: Can add unnecessary complexity for simple CRUD in MVC where direct `DbContext` usage is often sufficient and simpler.
- **Decision**: Depends on project size and team preference. For this course, we'll use direct `DbContext` in labs for simplicity, but acknowledge repository's role.

Repository Pattern (Adaptation for MVC)

- Interface

```
public interface IRepository<T> where T : class
{
    Task<IEnumerable<T>> GetAllAsync();
    Task<T> GetByIdAsync(int id);
    Task AddAsync(T entity);
    void Update(T entity); // Often not async as it just marks state
    void Delete(T entity);
    Task<int> SaveChangesAsync();
}
```

```
public class MovieRepository : IRepository<Movie>
 {
     private readonly ApplicationDbContext _context;

     public MovieRepository(ApplicationDbContext context)
     {
         _context = context;
     }

     public async Task AddAsync(Movie entity)
     {
         await _context.Movies.AddAsync(entity);
     }

     // ... implement other methods ...

     public async Task<int> SaveChangesAsync()
     {
         return await _context.SaveChangesAsync();
     }
 }
```

Assignment